Design and Analysis of a Secured Token-Bound Account

Sham Chun Ning Adrian[1], Hung Ka Fai Felix[2], and Liao Shih-wei[3]

[1,2,3]**Department of Computer Science and Information Engineering**
National Taiwan University
[1]**adriansham99@gmail.com**

Abstract

Non-Fungible Tokens (NFTs) have become a significant force since their rise in 2017, offering unique, ownable, and transferable digital assets. The latest advancement in this domain is the ERC6551 proposal, which introduces Token-Bounded Accounts (TBAs). By allowing NFTs to function as wallets, TBAs open new avenues for decentralized finance, gaming, on-chain identities, and more. However, TBAs also present security challenges, such as ownership cycles and malicious transfers, that must be addressed to ensure their safe and effective use. This paper explores the architecture and implementation of ERC6551, details the security concerns associated with TBAs, and proposes feasible solutions to mitigate these risks. While our proposed solutions show promise, further implementation, testing, and refinement are necessary.

Keywords: NFT, Token Bound Account, Blockchain, Solidity, Web3 Security

1. Introduction

The ever-evolving nature of Blockchain has provided the opportunity for developers to explore possibilities that are otherwise unachievable in web2 industries. Amongst the abundance of proposals and protocols, Non-Fungible Token (NFT) has been a prominent force within the industry since its rise in 2017 [1]. In essence, every NFT minted is unique, ownable, and transferrable. Because of the traits mentioned above, NFT allowed developers to design and create different representations of ownership, propelling the development of decentralized finance (DeFi), gaming, artwork, and many more decentralized applications (Dapp) within the Blockchain ecosystem. The latest improvement is the ERC-6551 proposal: Non-fungible Token-Bound Accounts (TBA) [2]. With the TBA proposal, users can turn any NFT into a wallet through the deployed registry contract, opening up new horizons for decentralized finance. This implementation is particularly useful in the following scenarios:

- Smart Wallet: TBA can integrate with ERC4337 Account Abstraction(AA) to achieve an NFT smart wallet [3]. The integration of AA allows the TBA to explore many more functionalities, including gasless transactions, account recovery, etc.
- Gaming: The integration of TBA into gaming opens up a world of possibilities where your characters can genuinely own NFTs. For instance, in developing a role-playing game (RPG), Developers can conceptualize the characters as TBAs, enabling gamers to collect items(NFTs) and store them in the character TBA like an inventory. Simultaneously, TBA characters can operate autonomously within the game, adding a new dimension to the gaming experience.
- On-chain identities: TBAs can serve as an on-chain agent to interact with services securely and transparently. Such digital assets can also represent identities, such as those in social apps. By using TBA, apps can facilitate micro-transactions much more efficiently, paving the way for a more seamless and user-friendly experience in social apps.

Although TBA has excellent potential, its protocol adoption rate is relatively low. According to the TBA official website, there are currently only 23 projects actively using TBA and 6 more projects from previous hackathon events [4]. When introducing new standards, resistance from the established community is expected. To encourage adoption among developers, marketplaces, and end-users, the benefits of TBA must be obvious and substantially outweigh the associated challenges [5].

The rest of the paper is organized as follows: Section 2 will go in-depth into related technical specifications; Section 3 will discuss the security considerations regarding the implementation of TBA. In Section 4, we will discuss and explore the possible solutions to help smoothen the adaptation of TBA in new protocols. In Section 5, we will provide and describe on our testing and evaluation. Lastly, in section 6, we will summarize the paper and provide future work.

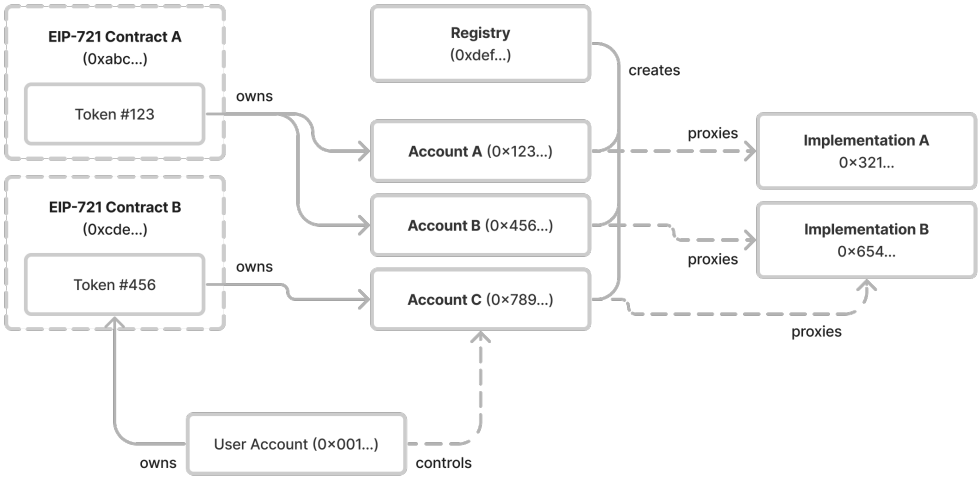

Fig. 1 Diagram of the architecture of ERC6551 [2]

2. Terms and Definitions

2.1 Blockchain

A Blockchain is a decentralized and distributed digital ledger that records transactions across multiple computers in a network that ensures security, transparency, and immutability [6]. Ethereum, a general-purpose Blockchain, features an embedded computer known as the Ethereum Virtual Machine (EVM). The EVM maintains a state everyone agrees upon on the Ethereum network. Application developers can upload smart contracts into the EVM state, and users make requests to execute these code snippets with varying parameters. Smart contracts are the programs uploaded to and executed by the network. Everything that runs on the Blockchain is, in essence, a smart contract. For example, every NFT is minted through an ERC721 smart contract, and every decentralized exchange (DEX) is designed, supported and maintained through deployments of one or more smart contracts.

2.2 NFT (ERC721)

NFTs are unique digital assets representing ownership of specific items, such as art, collectables, and virtual goods. The distinctive characteristics of Blockchain technology, particularly on platforms like Ethereum, facilitate the creation and management of NFTs. The Ethereum Blockchain supports the ERC721 standard [7], which defines a set of rules and interfaces for creating NFTs. This standard ensures that each token is unique and verifiable and provides the framework for transferring ownership and managing the properties of NFTs. Smart contracts allow developers to encode NFTs' properties and transfer mechanisms directly into the Blockchain using the ERC721 implementation. The decentralized nature of the Blockchain ensures that ownership records are transparent and immutable, preventing fraud and duplication. Additionally, the interoperability of the Ethereum network allows NFTs to be easily transferred and used across various platforms and applications, fostering a vibrant ecosystem for digital assets.

2.3 Token-Bound Account (ERC6551)

A Token-Bound Account (TBA) is a concept in Blockchain technology where an account's operations and permissions are tied to a specific token. This relationship means that the account's ability to interact with the Blockchain, execute transactions, or access certain features is contingent upon the ownership or presence of a particular token. ERC6551 is a proposed Ethereum standard that specifies the design and implementation of Token-Bound Accounts. It extends the capabilities of existing ERC standards, such as ERC721 for Non-Fungible Tokens (NFTs), by enabling these tokens to have their associated accounts with programmable logic.

Figure 1 provides an in-depth view of the architecture of ERC6551, illustrating the relationships between NFTs, NFT holders, token-bound accounts (TBAs), and the Registry. The Registry is a permissionless, immutable, and non-ownable singleton contract that is the entry point for all TBA address queries. To create a TBA, any user can invoke the `createAccount` function in the Registry. Each TBA is instantiated as a proxy contract of an implementation contract that inherits the `IERC6551Account` interface, which includes an `owner` function that always returns the owner of the target NFT. This implementation contract is customizable and defines the TBA's execution logic. As depicted in Figure 1, the Registry creates a token-bound Account C. Account C delegates its functions to Implementation Contract B and is owned by NFT Contract B - token

ID 456. Therefore, when a user owns NFT B - token ID 456, they gain the authority to control Account C. This setup ensures that the ownership and control mechanisms of TBAs are intrinsically linked to the NFTs, allowing for enhanced security and functionality within the Blockchain ecosystem.

3. Security Concerns

The ERC6551 proposal clearly states two security concerns: Ownership Cycles and Malicious Transfer. One of TBA's most significant advantages is the transferability of its ownership, allowing wallets that were initially not intended for transfer to be transferrable. However, such a perk is also the reason for its security concerns, as Blockchain technology does not yet have any implementation to safeguard such an action from malicious intent. We will delve deeper into both security concerns and discuss how transferring ownership of wallets could cause significant security concerns.

3.1 Ownership Cycles

Ownership cycles could occur when a TBA owns a particular NFT under the same NFT's ownership. The simplest form of ownership cycle can be represented by an NFT being transferred to its own TBA. In such a case, Both the NFT and all the assets in the TBA will become inaccessible, as an NFT cannot initiate a transaction. Ownership cycles can be introduced in any graph of $n \geq 1$ TBAs; an NFT owns a TBA that owns another TBA until the nth TBA owns the initial NFT.

Such a security concern is rather complicated to detect on-chain as the graph of TBAs can grow arbitrarily large. According to the number of vertices in the TBA graph, the number of searches for ownership cycles grows infinitely.

3.2 Malicious Transfer

A malicious transfer is when user A transfers an NFT X that owns TBA Y to user B, promising all the assets(tokens or NFTs) in TBA Y. However, upon completion, user B only receives X and Y, not the assets. Malicious actors can conduct such fraudulent behaviour in both a front-running and back-running manner.

1. Front-running: We can use the example in the ERC6511 proposal to demonstrate the scenario. Alice owns NFT X with TBA Y. She deposits 10 ETH into Y and places X on a marketplace for sale. Bob sees the NFT and offers 11 ETH to buy the NFT, expecting to get the 10 ETH stored in Y. Alice withdraws all 10 ETH from Y and accepts Bob's offer immediately. Bob has now used 11 ETH to buy X and an empty Y. As we can see, because the initial owner of the TBA has full access, they can withdraw all assets before transferring/selling the token to the receiver, deceiving them into paying more than they should.
2. Back-running: This case is very similar to the front-running case. However, the initial owner would pre-approve the tokens within the TBA instead of transferring the assets before completing the transfer. Therefore, once the transfer is complete, the initial owner can back-run the transaction by calling `transferFrom()` and effectively stealing away all the assets in the TBA.

As the above scenarios show, the mismatch in authority and information between the initial owner and receiver creates a critical security concern when establishing a trustless environment for exchanging TBAs.

4. Solution Design

This section will address the security concerns mentioned above with our suggested smart contract designs. The ERC6551 Proposal clearly states, "This proposal defines a system which assigns Ethereum accounts to all non-fungible tokens. These token-bound accounts allow NFTs to own assets and interact with applications, without requiring changes to existing smart contracts or infrastructure." Therefore, we are not looking for a solution through ERC721 modifications or extensions; instead, we are looking to design a more robust TBA smart contract template or standard for developers to adopt and preserve the proposal's backward compatibility.

4.1 Ownership Cycles Solution

Ownership cycles are difficult to detect or prevent because although the NFT and TBA are related, they are separate entities without direct interaction. Such an architecture complicates implementing a checking requirement during or before transferring an NFT. Let's say we want to implement a checking feature on the TBA smart contract whenever it receives an NFT. We could check whether the NFT received owns a TBA, and from there, we can check whether an ownership cycle would occur if the transfer were to go through. However, the problem is that the NFT does not record the TBA(s) it owns; in other words, the NFT itself or the NFT contract does not provide any information on whether or not it owns a TBA. As a result, it would be hard for the TBA smart contract to search for TBAs belonging to the NFT it received. It may need assistance from a third-party contract or extensive searching, resulting in lower transfer efficiency and higher gas costs. On the other hand, a bottom-up approach has similar issues. A TBA will not be able to know when the NFT it is bound to be transferred and has no authority over whether the transfer can succeed.

By acknowledging the complications in preventing ownership cycles, we suggest that instead of entirely preventing it from occurring, we can implement guardian accounts for account recovery in the case of its occurrence. We have previously designed a `guardianCompatible` module for TBA contracts to adopt [8]. It is used when ownership of the TBA is compromised; for example, if a user loses their wallet's private key, guardians can then vote to submit a transaction to transfer the NFT from the initial wallet to another functional wallet as an operator. A similar approach can be designed to solve ownership cycles.

```
abstract contract Guardian {
    #Any number of guardians
    address[] guardians = new address[](3);

    struct Recovery {
        uint approvalCount;
        uint startTime;
        mapping(address => bool) voted;
    }
    mapping() public recoveries;
    uint256 recoveryPeriod = 10 minutes;

    function addGuardian;
    function removeGuardian;
    function proposeChange;
    function voteChangeOwner;
    function executeChangeOwner;
}
```

Listing 1 Minimalist Coding Design of Guardian Implementation

All we need to do is set up a guardian and recovery voting systems within the TBA. Whenever assets become inaccessible, we can use guardians to perform a simple `transferFrom()` to break the cycle. Let's look at Figure 2 to get a better understanding.

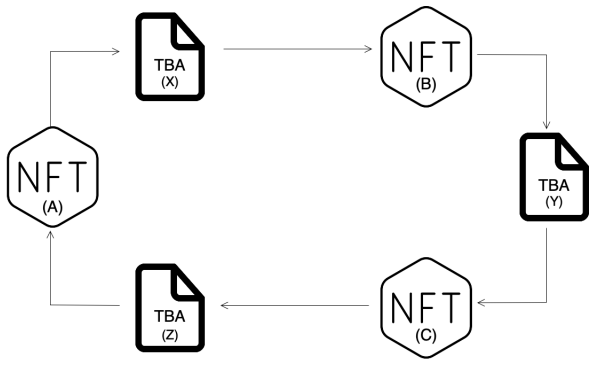

Fig. 2 Diagram of Ownership Cycle

From the diagram, we can see that the ownership cycle is represented by NFT A → TBA X → NFT B → TBA Y → NFT C → TBA Z → NFT A. Since in every ownership cycle, every TBA must own an NFT; therefore, as long as any of the TBAs transfers its corresponding NFT to another functional address, the ownership cycle will be broken. That is, if TBA X transfers NFT B to another address, Y transfers C or Z transfers A, ownership cycle will be broken.

4.2 Malicious Transfer Solution

The ERC6551 Proposal suggests that NFT marketplaces enforce safeguarding measures to prevent malicious transfers. Overreliance on NFT marketplaces limits the scalability and development of NFT, hindering its widespread adoption. The solutions suggested revolve around locking the assets during listing to prevent any front-run withdrawal actions. As discussed above, this solution only partially solves the issues as it cannot detect pre-approved assets, which may be vulnerable to rug-pull after transfer.

The best way to solve this problem is to modify and design a new standard for TBA contracts. One approach is to implement an array and store all token addresses approved by the smart contract. The whole design would look something like this.

```
contract TBA {
    address[] approvedTokens;
    address[] approvedNFTs;

    function approve;
    #Only approve through this function
    #Approvals will be pushed to \
    # approvedTokens or approvedNFTs

    function lock;
    #Lock all assets from transfer \
    #or withdraw and approve

    function resetApproval;
    #For loop approvedTokens and \
    #approvedNFTs and set to 0
}
```

Listing 2 Minimalist Coding Design of Approval Implementation

In general, the TBA could lock its own assets before a transfer to prevent front-running attacks. It can also reset its approval amount for every approved token to ensure a back-running rug-pull. This implementation raises the trustlessness of using TBAs and reduces the burden of NFT marketplaces when interacting with them.

5. Testing and Evaluation

To test the smart contracts' functionality, we used Foundry to run our smart contracts locally and produce testing results and data. We provide the source code on Github [9]. Our test results are documented in Figure 3.

```
Ran 7 tests for test/TBA.t.sol:TBATest
[PASS] test_approvalForAll() (gas: 390726)
[PASS] test_approveThroughExecute() (gas: 63630)
[PASS] test_backrunAttack() (gas: 299861)
[PASS] test_breakOwnershipCycle() (gas: 423783)
[PASS] test_createOwnershipCycle() (gas: 148944)
[PASS] test_frontrunAttack() (gas: 221980)
[PASS] test_lockAndReset() (gas: 388756)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 6.

Ran 1 test suite in 6.00s (6.00s CPU time): 7 tests passed, 0 f
```

Fig. 3 Foundry Test Results

The test suite evaluates the functionality and security of our solution design implemented TBA and their integration with a simple NFT marketplace. The setup function initializes the environment, deploys necessary contracts, and mints NFTs for the user. The `createOwnershipCycle` test demonstrates creating an ownership cycle involving multiple TBAs by transferring NFTs to form a circular ownership structure. The `breakOwnershipCycle` test confirms that guardians can break ownership cycles and recover assets by initializing guardians, creating a cycle, and executing recovery. The `approveThroughExecute` test validates the prevention of unauthorized approval transactions by attempting various approval-related actions and expecting reverts. The `approvalForAll` test tests managing token and NFT approvals within a TBA by setting operators, approving tokens and NFTs, and asserting stored approvals. The `frontrunAttack` and `backrunAttack` tests simulate front-running and back-running attack scenarios, respectively, to evaluate vulnerabilities and confirm the presence of security gaps. Lastly, the `lockAndReset` test demonstrates the effectiveness of the lock and reset mechanism in preventing unauthorized asset withdrawals during listings by depositing assets, locking the contract, resetting approvals, and simulating buyer interactions. The tests highlight key vulnerabilities, such as front-running and back-running attacks, and demonstrate the effectiveness of the lock and reset mechanism in enhancing security during listings.

6. Conclusion and Future Works

Fungible Tokens (NFTs) have emerged as a significant force, enabling unique, ownable, and transferable digital assets. The ERC6551 proposal, which introduces Token-Bounded Accounts (TBAs), represents a significant advancement in the blockchain ecosystem. By allowing NFTs to function as wallets, TBAs open new possibilities for decentralized finance, gaming, on-chain identities, and more.

Despite their potential, TBAs face several security challenges, notably ownership cycles and malicious transfers. Ownership cycles, where a TBA owns an NFT that indirectly owns itself, can render assets inaccessible and are challenging to detect due to the potentially vast and complex graph of TBAs. Malicious transfers can deceive users by transferring NFTs without their associated assets, exploiting gaps in the transaction process.

Addressing these concerns requires a multifaceted approach. Our solution design is feasible and offers a promising approach to addressing the identified security concerns. However, it is important to note that there are limitations and certain aspects may require further implementation, testing, and refinement. Implementing guardians into TBA contracts will deprive it of being entirely bound. Guardians will have fractional ownership or authority, which is against the motivation of the ERC6551 proposal. Whether or not giving away minor authority is worth recovering ownership is still up for further discussion. Also, implementing an array for all asset approvals may be costly. For each approval, additional execution resources would be used to record and store data. Additionally, whenever NFTs have to be sold, calling lock and reset requires additional execution resources. If the arrays grow arbitrarily large, iteration and operation may become costly. Such a solution may be feasible but not optimal. Continued evaluation and iterative development will be crucial to ensure the robustness and effectiveness of our solution.

Future work includes researching into optimizing the balance between decentralization and the implementation of guardians or other recovery mechanisms will also be crucial, ensuring that TBAs maintain their intended autonomy while providing safety nets for asset recovery. Additionally, further investigation into cost-effective methods for managing asset approvals and transaction locking is needed to prevent front-running and back-running attacks without imposing excessive resource demands. Exploring alternative approaches or enhancements to the current design could yield more efficient and scalable solutions. Collaboration with the broader blockchain community, including developers, researchers, and users, will be vital to gather feedback, identify new use cases, and address emerging challenges. Iterative testing, real-world deployments, and comprehensive security audits will help refine the proposed solutions and ensure their robustness. Ultimately, continuous innovation and refinement of the ERC6551 standard will pave the way for more secure, functional, and user-friendly TBAs, driving their adoption and unlocking new possibilities within the blockchain ecosystem.

References

[1] Jolene Creighton. “NFT Timeline: The Beginnings and History of NFTs”. In: *nftnow* (2022). URL: `https://nftnow.com/guides/nft-timeline-the-beginnings-and-history-of-nfts/`.

[2] Jayden Windle et al. *ERC-6551: Non-fungible Token Bound Accounts*. URL: `https://eips.ethereum.org/EIPS/eip-6551#fraud-prevention`.

[3] Viktor Valaštín et al. *Towards Proxy Staking Accounts Based on NFTs in Ethereum*. 2024. arXiv: `2404.14074 [cs.DC]`. URL: `https://arxiv.org/abs/2404.14074`.

[4] URL: `https://docs.tokenbound.org/showcase`.

[5] Adam Zion. *Unlocking New Dimensions for NFTs with ERC-6551 'Backpack Wallet'*. URL: https://www.dynamic.xyz/blog/erc-6551#:~:text=Ownership%20Cycles%3A%20There's%20a%20risk,its%20assets%20become%20permanently%20inaccessible..

[6] Corwin Smiths et al. "INTRO TO ETHEREUM". In: *ethereum.org* (2023). URL: https://ethereum.org/en/developers/docs/intro-to-ethereum/#what-is-a-blockchain.

[7] William Entriken et al. *ERC-721: Non-Fungible Token Standard*. URL: https://eips.ethereum.org/EIPS/eip-721.

[8] Sham Chun Ning Adrian. *Token-Bound-Account-Project*. https://github.com/shamadrian/Token-Bound-Account-Project. 2023.

[9] Sham Chun Ning Adrian. *Secure-TBA-Design*. https://github.com/shamadrian/Secure-TBA-Design/tree/master. 2024.